Hypermedia Applicaton
Technology Report

Yuto Yamaguchi yuto.yamaguchi@mail.polimi.it
Xavier Anicito xavierandre.anicito@mail.polimi.it
Ewan Decima ewangabriel.decima@mail.polimi.it

April 2025

Contents

1 Abstract

The purpose of this technical document is to provide a comprehensive overview of the technical decisions made prior to the development process of the *Yoga center* website. It meticulously details the project's architecture and the components meticulously designed and developed for its functionality. Furthermore, the document includes a comprehensive list of additional modules and libraries employed, elucidating the meticulous process through which the website was ensured to adhere to accessibility and SEO guidelines.

2 Introduction

2.1 Team

The present work was conducted by the **WeAreLookingForUserInterfaceDesigner** team, consisting of

- Yuto Yamagachi -
- Xavier Anicito -
- Ewan Decima - 11109703

2.2 Project

All relevant project materials—including the source code, design mockups, and deployed version—can be accessed via the following links:

- Website: HMWA-25.vercel
- GitHub: github.com/moise7000/HMWA-25
- Figma: HMWA-25.figma

2.3 Work breakdown

In an effort to maintain a balanced workload among all members, ensuring equal distribution and accessibility, we aimed to facilitate comprehensive learning during this project. However, to expedite our progress, we made the necessary adjustments by dividing the workload.

Specifically, while Xavier Anicito and Yuto Yamagachi concentrated on the high wireframes, Ewan Decima focused on database schema. Despite these changes, all members contributed to the design and implementation of the application's components.

3 Documentation

3.1 Theme

We have decided to develop a website for *Yoga center*, a yoga and wellness center located in Milan, Italy.

3.2 Technology

In order to meet both the functional and technical requirements of the project, we carefully selected the following technologies and approaches:

- *Server-Side Development*: We implemented our backend using **Nuxt**, adhering to the project's requirement.

- *Application Hosting*: We selected **Vercel** as our hosting platform over GitHub Pages. Vercel offers more advanced features than GitHub Pages, especially for modern web development. It supports dynamic frameworks like Nuxt, automatic deployments with preview URLs for each branch or pull request, built-in serverless functions. In contrast, GitHub Pages is limited to static sites and requires more manual setup.
- *Database Hosting*: We utilized **Supabase**, which provides a PostgreSQL-based relational model. This choice was natural, as the data encompasses multiple interrelated entities. Additionally, Supabase was used to handle user authentication, offering a seamless and secure way to manage sign-up, login, and session management.
- *Scripting Language*: We adopted **TypeScript** over JavaScript to enhance code robustness, maintainability, and scalability. TypeScript's static typing allowed us to define precise types for our database entities, and UI components, which significantly improved the developer experience. It also enabled better tooling support, safer refactoring, and early detection of potential bugs, ultimately reducing runtime errors and improving overall code quality.

3.3 Structure

3.3.1 Pages

Page	**URL**	**Description**
Home	/	It is *Yoga center*'s home page. It summarizes content from other pages and provides various links to them.
About	/about	
Benefits	about/benefits	
Yoga equipments	/about/equipments	
Practice at home	/about/practice-at-home	
Articles	/about/articles	
Article	/about/articles/[slug]	
Contact	/contact	
Institute	/institute	
Story	/institute/story	
Teachers	/institute/teachers	
Teacher	/institute/teachers/[slug]	
Location	/institute/location	
Events	/events	
Event	/events/[slug]	
Courses and Subscriptions	/courses-and-subscriptions	
Courses	/courses-and-subscriptions/courses	
Course	/courses-and-subscriptions/courses/[slug]	
Subscriptions	/courses-and-subscriptions/subscriptions	

3.4 Custom types

In the web application, we implemented TypeScript types to establish a robust and consistent connection with the data stored in Supabase. By defining types that correspond to the structure of the database tables, we ensure type safety throughout the application, thereby mitigating the risk of runtime errors and enhancing code reliability. Furthermore, TypeScript's object-oriented approach contributes to the codebase's readability and organization, facilitating both development and future maintenance efforts. For instance We created

- `Article`: represent an yoga article

- `Course`: represent an yoga course offers by the center
- `CourseEnrollement`: represent the relation between a course and a student
- ...

graphicx

3.5 Components

The majority of our website's components are custom-designed and developed with four fundamental principles in mind:

- *Reusability*: Numerous elements, such as cards and buttons, are utilized throughout the site. To ensure consistency and prevent code duplication, these elements were encapsulated into reusable components.
- *Modularity*: Component-based development significantly enhances code readability. Even when a component is utilized solely once, defining it separately contributes to cleaner, more maintainable code.
- *Consistency*: We made a deliberate effort to maintain visual and structural consistency across components. This encompasses unified naming conventions, layout patterns, and styling approaches to ensure a cohesive user experience throughout the application.
- *Simplicity*: We endeavored to maintain our components as simple and intuitive as possible. By reducing complexity, developers can more readily comprehend, utilize, and modify components, thereby facilitating faster development and reducing the learning curve for new contributors.

The following section provides a comprehensive overview of the custom components we have implemented.

3.5.1 Cards

`CourseCard`

This component is of paramount importance and serves as the central element in the context of a yoga center, as it visually represents the courses offered by the center. Therefore, it is imperative that it be meticulously designed.

Prop	Type	Required	Description
`course`	`Course`	yes	a yoga course

To provide a comprehensive understanding of the `CourseCard` component, it is essential to delve into the specific parameters that define a yoga course. Below is a detailed breakdown of the main properties associated with the Course object:

Prop.attribute	Type	Description
`course.image`	`string | null`	the path of the image associated with the yoga course stored in Supabase
`course.title`	`string`	the title of the yoga course
`course.difficulty_level`	`string`	beginner - intermediate - advanced or all levels, defining the difficulty of the course
`course.teacher`	`Teacher`	the teacher leading the course
`course.description`	`string`	description of the course
...		

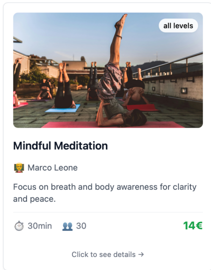

Figure 1: `CourseCard`

`ArticleCard`

This component displays a card for a given article, including its title, a brief excerpt of its content, and the date of publication. Prior to `CourseCard`, the first table of is relatively straightforward.

Prop	Type	Required	Description
`article`	`Article`	yes	A yoga article

And more precisely

Prop.attribute	Type	Description
`article.image`	`string | null`	the path of the image associated with the yoga article stored in Supabase
`article.title`	`string`	the title of the yoga article
`article.published_at`	`string`	the publication date of the article
`article.content`	`string`	a snippet of the article's content (maximum 250 characters)

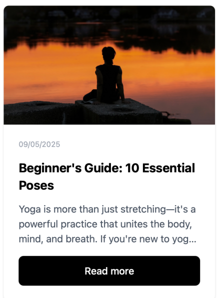

Figure 2: `ArticleCard`

`BenefitCard`

Prop	Type	Required	Description
`title`	`string`	yes	a the title of the benefit
`description`	`string`	yes	a the description of the benefit
`icon`	`string | null`	false	an icon to symbolize the benefits (we use emojis but you can use path to static image)

Figure 3: `BenefitCard`

`EquipmentModal`

This component displays a card for a given equipment, including its name, a brief description, and a link to see where to buy it. Prior to `CourseCard`, the first table of is relatively straightforward.

Prop	Type	Required	Description
`equipment`	`Equipment`	yes	A yoga equipment

And more precisely

Prop.attribute	Type	Description
`equipment.image`	`string | null`	the path of the image associated with the yoga equipment stored in Supabase
`equipment.name`	`string`	the name of the yoga equipment
`equipment.description`	`string`	the publication date of the article
`equipment.purchase_link`	`string`	a string that represents the URL of the partner's website

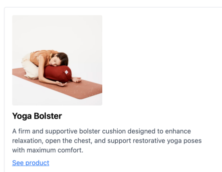

Figure 4: `EquipmentCard`

TeacherCard

This component displays a card for a given teacher, including his name, a brief bio some of his certifications. Prior to `CourseCard`, the first table of is relatively straightforward.

Prop	Type	Required	Description
`teacher`	`Teacher`	yes	A yoga teacher

And more precisely

Prop.attribute	Type	Description
`teacher.photo`	`string | null`	the path of the yoga teacher's profile photo stored in Supabase
`teacher.name`	`string`	the name of the yoga teacher
`teacher.biography`	`string`	the biography of the teacher
`teacher.certificates`	`string[]`	a list of yoga certifications

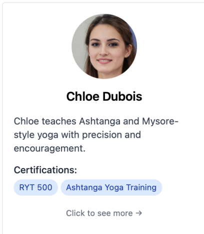

Figure 5: `teacherCard`

FeedbackCard

This component presents a card for a specific feedback, encompassing the name of the student who authored it, the comment, and the date of publication. Prior to `CourseCard`, the first table of is relatively straightforward.

Prop	Type	Required	Description
`feedback`	`Feedback`	yes	A yoga feedback

And more precisely

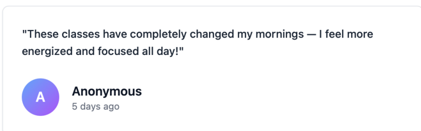

Figure 6: `feedbackCard`

3.5.2 Navigation

Navbar

This component serves as the navigation bar of the web application, encompassing all its key landmarks. Its layout undergoes a significant transformation in accordance with the maximum available screen width.

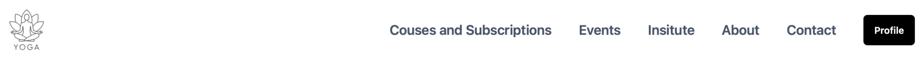

Figure 7: `Navbar`

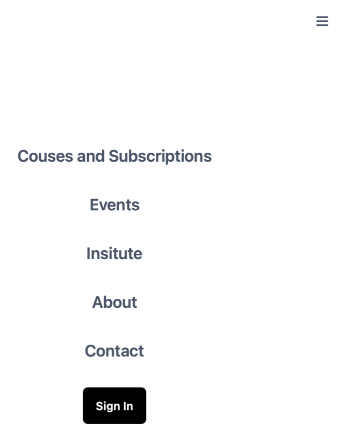

Figure 8: `Navbar for mobile devices`

`Footer`

This component serves as the footer of the web application, providing a comprehensive list of landmarks and essential links.